

 UNIVERSIDAD DE GRANADA Dpto. LSI	Programación y Diseño Orientado a Objetos (PDOO) Grado en Ingeniería Informática Evaluación Continua: Segundo Parcial de Teoría (Solución propuesta)
--	---

Pregunta 1. [2,5 puntos] Responda verdadero (V) o falso (F) a las siguientes preguntas sobre fundamentos teóricos de PDOO. Sea verdadera o falsa su respuesta, solo tendrá que justificarla en los casos habilitados para ello. *Justificaremos todas por cuestiones docentes.*

N	Pregunta	V/F
1	Una clase abstracta en Java puede ser instanciada directamente con la palabra clave new .	F
	<i>Justificación: Las clases abstractas en Java no se pueden instanciar con new, solo a través de instanciar objetos de clases hijas.</i>	
2	La vinculación (ligadura) estática en Java ocurre en tiempo de ejecución.	F
	<i>Justificación: la ligadura estática hace referencia al tiempo de compilación; y la dinámica, al tiempo de ejecución.</i>	
3	El método clone() en Java realiza una copia profunda por defecto.	F
	<i>Justificación: El método clone() por defecto realiza una copia de campos primitivos y referencias, pero en el caso de objetos mutables y colecciones, se deben clonar explícitamente para lograr una copia profunda.</i>	
4	Una clase en Java puede implementar múltiples interfaces y heredar de múltiples clases a la vez.	F
	<i>Justificación: Una clase en Java puede implementar múltiples interfaces pero solo puede heredar de una clase.</i>	
5	Sobrescribir un método en Java puede cambiar su tipo de retorno.	F
	<i>Justificación: Al sobrescribir un método se debe mantener el mismo tipo de retorno.</i>	
6	Todos los miembros de una interfaz son públicos y finales por defecto.	F
	<i>Justificación: Todos los miembros de una interfaz son públicos, pero no necesariamente finales por defecto; pero, sí son abstractos por defecto.</i>	
7	El polimorfismo en Java no permite que objetos de una subclase sean tratados como objetos de su superclase.	F
	<i>Justificación: El polimorfismo permite que objetos de una subclase sean tratados como objetos de su superclase.</i>	
8	La palabra clave final en Java puede aplicarse a clases abstractas.	F
	<i>Justificación: Las clases abstractas no pueden ser finales porque están destinadas a ser extendidas.</i>	
9	La encapsulación en Java se logra únicamente a través del uso de getters y setters.	F
	<i>Justificación: hay muchos mecanismos de encapsulación además de getters y setters: por ejemplo, definir una Interfaz con los métodos que tendremos que implementar en el futuro, pero sin conocer cómo serán.</i>	
10	Una clase hija en Java no es propietaria de los atributos privados del padre, por ello usamos <i>private</i> .	F

Pregunta 2. [2,5 puntos] Responda verdadero (V) o falso (F) a las siguientes preguntas sobre el diagrama de clases. Sea verdadera o falsa su respuesta, solo tendrá que justificarla en los casos habilitados para ello. *Justificaremos todas por cuestiones docentes.*

N	Pregunta	V/F
1	El método recolectar en la clase Parcela debe ser sobrescrito obligatoriamente en todas sus subclases. <i>Justificación: El método recolectar no está marcado como abstracto, por lo tanto, no hay obligatoriedad de sobrescribirlo en las subclases.</i>	F
2	Al implementar el método clone() en la clase Cultivo para realizar una copia profunda, es necesario clonar explícitamente el atributo riego y el atributo enumerado TipoCultivo para asegurar que la copia sea realmente profunda. <i>Justificación: No es necesario clonar explícitamente los atributos primitivos o inmutables como riego (que es de tipo primitivo int) o enumerados como TipoCultivo. Los tipos primitivos son copiados directamente, y los enumerados son inmutables y constantes por definición en Java, por lo que pueden ser compartidos entre diferentes instancias sin que afecte la independencia de las copias. El método clone() por defecto realiza una copia de campos primitivos y referencias, pero en el caso de objetos mutables y colecciones, se deben clonar explícitamente para lograr una copia profunda —lo que no es el caso por las razones anteriormente comentadas.</i>	F
3	La clase Controlador utiliza el patrón <i>Singleton</i> , esto implica que no puede tener atributos de instancia. <i>Justificación: El patrón singleton asegura que solo exista una instancia de la clase Controlador.</i>	F
4	La relación entre Cultivo y Soja lo que significa es que Soja puede existir independientemente de Cultivo . <i>Justificación: Soja es un tipo de Cultivo, por lo que no se trata de una relación que denote independencia.</i>	F
5	Dado que Producto es una clase abstracta, las clases que heredan de ella como Soja y Trigo pueden ser instanciadas si no implementan todos los métodos abstractos de Producto . <i>Justificación: Soja y Trigo pueden ser instanciadas si proporcionan implementaciones concretas de los métodos abstractos de Producto; sino, tienen que mantenerse abstractas y no se podrán instanciar.</i>	F
6	Dado que Producto es una clase abstracta y Cultivo —que hereda de Producto— también es abstracta, viendo solamente el diagrama no podemos saber por qué continúa siendo abstracta Cultivo . <i>Justificación: Cultivo continúa siendo abstracta porque debe tener, al menos, un método abstracto. En este caso, el único método abstracto que sabemos que debe tener es «enProduccion()», heredado de Producto. Como «enProduccion()» no aparece en el diagrama de Cultivo, como sí que aparece en Soja y Trigo, sabemos que ese al menos sigue siendo abstracto en Cultivo, por eso es clase abstracta.</i>	F
7	En la clase Trigo , vemos el método enProduccion() , pero en Cultivo no lo vemos (abstracto), esto implica que Trigo no proporciona ninguna implementación específica de este método y no puede ser instanciada. <i>Justificación: al contrario, cuando ya aparece en el diagrama el método en cuestión (sin cursivas, ni <<abstract>>), es que ya se ha implementado, y por tanto Trigo se puede instanciar (porque no tiene ningún método abstracto más sin implementar que podamos saber con el diagrama).</i>	F
8	Dado que la clase Soja extiende de la clase abstracta Cultivo , cualquier método de Cultivo sobrescrito en Soja utilizará ligadura estática para resolver a qué implementación llamar cuando se invoque el método. <i>Justificación: En Java, la sobrescritura de métodos siempre utiliza ligadura dinámica, lo que significa que el método a ejecutar se determinará en tiempo de ejecución basándose en el tipo real del objeto, no en el tipo de referencia. Esto permite el polimorfismo y es uno de los principios fundamentales de la programación orientada a objetos en Java.</i>	F

9	La ligadura dinámica en Java impide que el método actualizarVista() de la clase VistaParcela sea invocado dinámicamente en tiempo de ejecución si el método fue declarado en la interfaz IVista .	F
	<i>Justificación: VistaParcela no implementa la interfaz IVista, la relación entre VistaPrincipal y VistaParcela es de dependencia, es decir que usa a VistaParcela, quien tiene un método actualizarVista() que nada tiene que ver con el que se declara en la interfaz IVista.</i>	
10	Según el patrón Modelo-Vista-Controlador, la clase Controlador debe contener lógica de negocio, como el cálculo de progresos y niveles, ya que actúa como intermediaria entre la vista y el modelo.	F
	<i>Justificación: es el Modelo quien tiene la lógica de negocio.</i>	

Pregunta 3. [2,5 puntos] Dado el siguiente código que intenta implementar el método **clone()** en la clase **Parcela** para realizar una copia profunda de sus objetos, identifica y corrige los errores para que el método funcione según lo requerido por el diagrama UML proporcionado:

```
public class Parcela implements Cloneable {
    private List<Cultivo> cultivos;
    private static final int MAX_CULTIVOS = 1;
    private static final int CAPACIDAD = 2;

    public Parcela() {
        this.cultivos = new ArrayList<>();
    }

    @Override
    public Parcela clone() throws CloneNotSupportedException {
        Parcela parcelaClonada = (Parcela)super.clone();
        parcelaClonada.cultivos = this.cultivos; //😬 Clonación superficial, incorrecta.
        return parcelaClonada;
    }
    //...resto de métodos no interesantes para el ejercicio...
}
```

Su corrección propuesta (puede indicar en comentarios si va a mantener algo del código suministrado, para ahorrar):

```
public class Parcela implements Cloneable {
    /*...  otros métodos y atributos  ...*/
    // Método clone corregido para realizar una copia profunda
    @Override
    public Parcela clone() throws CloneNotSupportedException {
        Parcela parcelaClonada = (Parcela)super.clone();
        parcelaClonada.cultivos = new ArrayList<>();
        for (Cultivo c : this.cultivos) {
            parcelaClonada.cultivos.add(c.clone());
        }
        return parcelaClonada;
    }
}

public abstract class Cultivo implements Cloneable { // 🎁
    private int riego; // Atributo primitivo, no necesita clonación profunda // 🎁
    private TipoCultivo tipo; // Atributo enumerado, tampoco necesita // 🎁

    //...suponemos constructores y métodos correctamente implementados...
    // y vamos a lo que importa:

    @Override
    public Cultivo clone() throws CloneNotSupportedException{
        return (Cultivo) super.clone();
    }
}
```

Pregunta 4. [2 puntos] Suponga que tiene un array de tipo **Cultivo** llamado **huerto** que almacena **referencias** a objetos de tipos que son subclases de **Cultivo**. El array **huerto** ya contiene instancias de **Soja** y **Trigo**. Asimismo, la subclase **Trigo** tiene un método único **abonar()** que no está presente en la clase **Soja**. Complete el siguiente fragmento de código para recorrer el array **huerto**, y que se empiece a producir cada tipo de cultivo y, además, si el cultivo es de tipo **Trigo**, que lo abone:

```
Cultivo[] huerto = { new Soja(), new Trigo(), new Trigo() }; // 📦
for ( Cultivo cultivo : huerto ) { // A ver qué tipo de for escribe aquí 😊

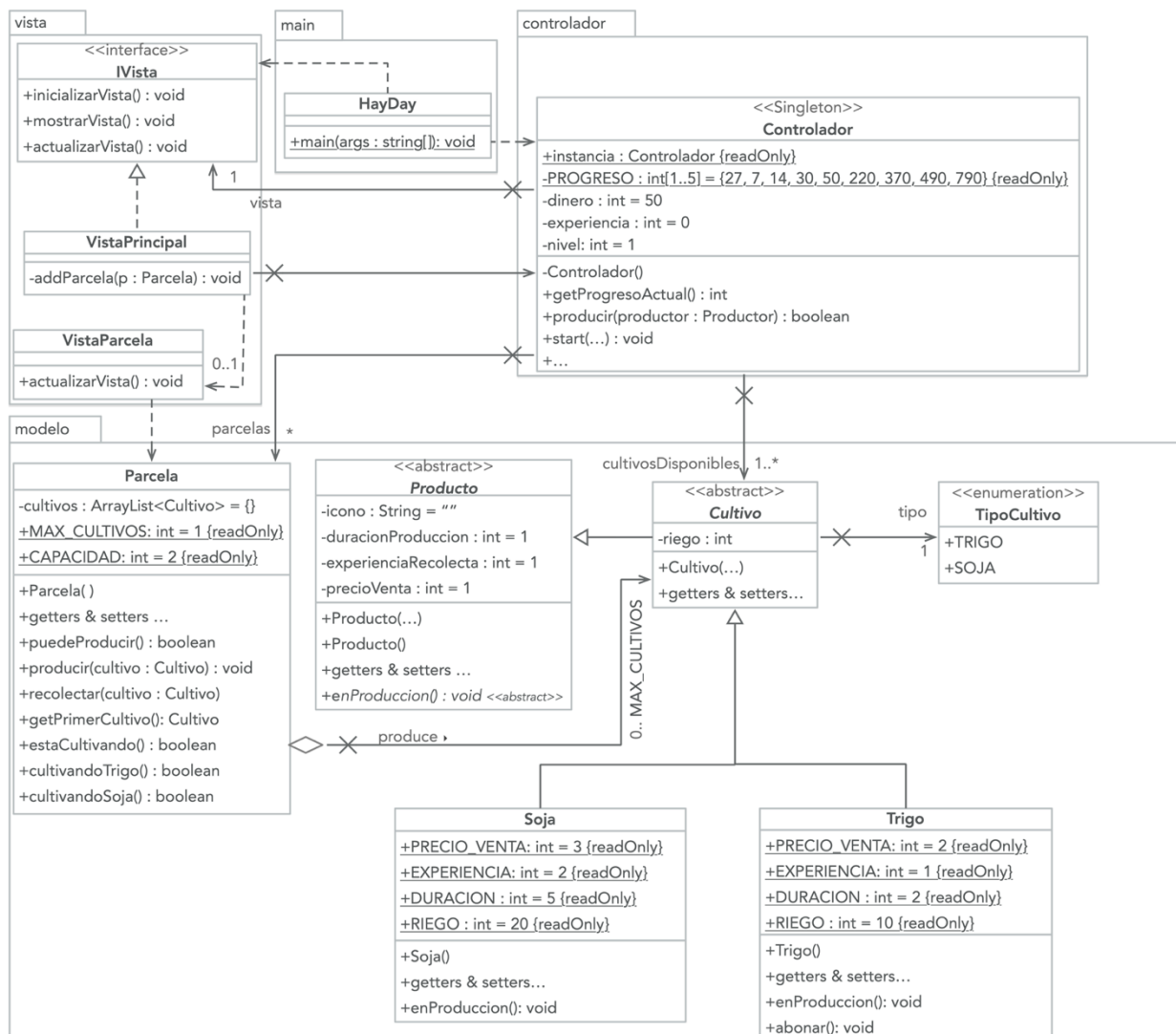
    cultivo.enProduccion();

    if (cultivo instanceof Trigo) {
        ((Trigo) cultivo).abonar(); //casting
    }

}
```

Pregunta 5. [0,5 puntos] ¿Qué sucedería si se intenta llamar a **abonar()** en una referencia de tipo **Cultivo** sin haber sido implementado en las subclases?

*Si se intenta llamar al método **abonar** en una referencia de tipo **Cultivo** el cual no tiene dicho método, sino que lo tiene solamente su clase hija **Trigo**, se producirá un error de compilación.*



Pregunta Extra (solo si se ha completado todo el examen). [🎁 puntos] Programa algo divertido para el juego con libertad; o un sistema de Trucos; Logros... Usa conceptos de herencia, polimorfismo, ligadura dinámica, etc.

```
abstract class Logro {
    protected String nombre;

    public Logro(String nombre) {
        this.nombre = nombre;
    }

    public abstract boolean verificarLogro(Jugador jugador);
}

class LogroCosechador extends Logro {
    private int cantidadNecesaria;

    public LogroCosechador(String nombre, int cantidadNecesaria) {
        super(nombre);
        this.cantidadNecesaria = cantidadNecesaria;
    }

    @Override
    public boolean verificarLogro(Jugador jugador) {
        return jugador.getCantidadCosechada() >= cantidadNecesaria;
    }
}

class LogroConstructor extends Logro {
    private int cantidadNecesaria;

    public LogroConstructor(String nombre, int cantidadNecesaria) {
        super(nombre);
        this.cantidadNecesaria = cantidadNecesaria;
    }

    @Override
    public boolean verificarLogro(Jugador jugador) {
        return jugador.getParcelasConstruidas() >= cantidadNecesaria;
    }
}

class RegistroLogros {
    private List<Logro> logros;

    public RegistroLogros() {
        logros = new ArrayList<>();
        // Agregar logros al registro
        logros.add(new LogroCosechador("Experto Cosechador", 100));
        logros.add(new LogroConstructor("Maestro Constructor", 5));
    }

    public void verificarLogros(Jugador jugador) {
        for (Logro logro : logros) {
            if (logro.verificarLogro(jugador)) {
                System.out.println("¡Has conseguido el logro: " + logro.nombre + "!");
            }
        }
    }
}
```

Si te apasiona programar o si quieres aprender más de sobre videojuegos u otras cuestiones, te espero para acompañarte en tu futuro TFG 🤖